

207. Course Schedule Medium

Acceptance: 50.6% | [LeetCode](#)

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

```
**Input:** numCourses = 2, prerequisites = [[1,0]]
**Output:** true
**Explanation:** There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.
```

Example 2:

```
**Input:** numCourses = 2, prerequisites = [[1,0], [0,1]]
**Output:** false
**Explanation:** There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.
```

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are **unique**.

Claude Opus Solution

Course Schedule — Cycle Detection in a Directed Graph

1. Intuition

Key Insight: This problem is fundamentally about **detecting a cycle in a directed graph**.

- Each course is a **node**.
- Each prerequisite `[a, b]` (take `b` before `a`) is a **directed edge** `b → a`.
- You can finish all courses **if and only if** there's no circular dependency.

A circular dependency (e.g., A needs B, B needs A) means you can never start either course. In graph terms, this is a **cycle**.

So the problem reduces to: **Is this directed graph a DAG (Directed Acyclic Graph)?** - If yes → return `True` (you can do a valid topological ordering). - If it has a cycle → return `False`.

There are two classic ways to detect cycles: 1. **DFS with 3-color marking** (white/gray/black). 2. **BFS / Kahn's Algorithm** (topological sort via in-degrees).

2. Approach

Approach A: BFS (Kahn's Algorithm) — Most Intuitive

1. Build adjacency list and in-degree count for every node.
 - For edge `b → a`: `graph[b].append(a)`, `indegree[a] += 1`
2. Add all nodes with `indegree == 0` to a queue (courses with no prerequisites).
3. Process the queue:
 - Pop a node, increment "completed courses" counter.
 - For each neighbor, decrement its indegree.
 - If a neighbor's indegree becomes `0`, add it to the queue.
4. If we processed all `numCourses` nodes → no cycle → return `True`. Otherwise → some nodes stuck in a cycle → return `False`.

Approach B: DFS with 3-Color Marking

- WHITE (0): unvisited
- GRAY (1): currently in the recursion stack (being explored)
- BLACK (2): fully explored

If during DFS we reach a GRAY node, we found a back-edge → cycle exists.

3. Python Solutions

Solution A: BFS / Kahn's Algorithm (Recommended)

```
from collections import defaultdict, deque
from typing import List

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        # graph[b] = list of courses that depend on b (edge b -> a)
        graph = defaultdict(list)
        # indegree[a] = number of prerequisites course 'a' still needs
        indegree = [0] * numCourses

        # Build the graph: prerequisite [a, b] means b must come before a (b -> a)
        for a, b in prerequisites:
            graph[b].append(a) # b unlocks a
            indegree[a] += 1    # a has one more prerequisite

        # Start with all courses that have no prerequisites (indegree 0)
        queue = deque(course for course in range(numCourses) if indegree[course] == 0)

        # Count how many courses we successfully "take"
        completed = 0

        while queue:
            course = queue.popleft()
            completed += 1 # We can take this course now

            # Taking this course may unlock other courses
            for next_course in graph[course]:
                indegree[next_course] -= 1 # one prerequisite satisfied
                # If all prerequisites are satisfied, it's ready to take
                if indegree[next_course] == 0:
                    queue.append(next_course)

        # If we managed to take every course, there's no cycle
        return completed == numCourses
```

Solution B: DFS with 3-Color Marking (Recursive)

```
from collections import defaultdict
from typing import List

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        graph = defaultdict(list)
        for a, b in prerequisites:
            graph[b].append(a) # edge b -> a

        # 0 = unvisited, 1 = in current DFS path, 2 = fully processed
        state = [0] * numCourses

        def has_cycle(node: int) -> bool:
            if state[node] == 1: # Revisiting a node on current path => cycle
                return True
            if state[node] == 2: # Already verified safe, skip
                return False

            state[node] = 1 # Mark as "being explored"
            for neighbor in graph[node]:
                if has_cycle(neighbor):
                    return True
            state[node] = 2 # Done exploring; mark safe
            return False

        # Check every node (graph may be disconnected)
        for course in range(numCourses):
            if has_cycle(course):
                return False
        return True
```

```
        return False
    return True
```

4. Complexity Analysis

For both approaches, where $V = \text{numCourses}$ and $E = \text{len}(\text{prerequisites})$:

Metric	Complexity	Explanation
Time	$O(V + E)$	We build the graph in $O(E)$, and each node and edge is visited exactly once during traversal.
Space	$O(V + E)$	Adjacency list stores $O(E)$ edges; indegree array / state array and the queue/recursion stack use $O(V)$.

Why $O(V + E)$ and not $O(V \cdot E)$? In Kahn's algorithm, each node enters the queue at most once (V operations), and each edge is examined exactly once when its source is processed (E operations).

5. Alternative Approaches

Approach	Notes
BFS / Kahn's Algorithm	Best for this problem. Naturally produces a topological order as a bonus. Avoids recursion depth limits.
DFS (recursive, 3-color)	Clean and elegant, but risks stack overflow for deep graphs (Python default recursion limit ~ 1000 ; here $V \leq 2000$, so use iterative or raise limit).
DFS (iterative, 3-color)	The reference solution — avoids recursion limits by managing an explicit stack. Slightly trickier to read.
Union-Find	✗ Not suitable here. Union-Find detects cycles in undirected graphs but cannot distinguish directed cycles from valid DAG diamond shapes (e.g., $a \rightarrow b$, $a \rightarrow c$, $b \rightarrow d$, $c \rightarrow d$).

Recommendation: Use **Kahn's Algorithm (BFS)** — it's iterative (no recursion limit issues), intuitive (mirrors "take courses as prerequisites get satisfied"), and gives you a topological order for free if needed (relevant for the follow-up problem [210. Course Schedule II](#)).

Codex Solution

1. Intuition

This problem is asking whether all courses can be completed given prerequisite relationships.

A prerequisite pair:

```
[a, b]
```

means:

```
To take course a, you must first take course b
```

So we can model this as a **directed graph**:

```
b -> a
```

If the graph contains a **cycle**, then it is impossible to finish all courses.

Example:

```
0 -> 1
1 -> 0
```

Course `0` requires `1`, and course `1` requires `0`, so neither can be taken first.

Therefore, the key insight is:

We need to check whether the directed graph is a DAG: Directed Acyclic Graph.

A common way to do this is **topological sorting** using **Kahn's Algorithm**.

2. Approach

We use **BFS Topological Sort**.

Key idea

Courses with 0 prerequisites can be taken immediately.

After taking such a course, we can reduce the prerequisite count of the courses that depend on it.

If eventually we can take all courses, return `True`.

If some courses are still blocked, there must be a cycle, so return `False`.

Pseudocode

```
function canFinish(numCourses, prerequisites):  
    create adjacency list graph  
    create indegree array of size numCourses initialized to 0  
  
    for each [course, prereq] in prerequisites:  
        add course to graph[prereq]  
        indegree[course] += 1  
  
    create queue  
  
    for each course from 0 to numCourses - 1:  
        if indegree[course] == 0:  
            add course to queue  
  
    completed_courses = 0  
  
    while queue is not empty:  
        current = queue.pop()  
  
        completed_courses += 1  
  
        for each next_course in graph[current]:  
            indegree[next_course] -= 1  
  
            if indegree[next_course] == 0:  
                add next_course to queue  
  
    return completed_courses == numCourses
```

3. Python Solution

```
from collections import deque  
from typing import List  
  
class Solution:  
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:  
        # graph[course] stores all courses that depend on this course  
        #  
        # Example:  
        # prerequisite [1, 0] means:  
        # take course 0 before course 1  
        #  
        # So we create an edge:  
        # 0 -> 1  
        graph = [[] for _ in range(numCourses)]  
  
        # indegree[i] = number of prerequisites course i still needs  
        indegree = [0] * numCourses  
  
        # Build graph and indegree array  
        for course, prereq in prerequisites:  
            # prereq must be taken before course  
            graph[prereq].append(course)
```

```

    # course has one more prerequisite
    indegree[course] += 1

# Queue stores all courses that currently have no prerequisites
queue = deque()

for course in range(numCourses):
    if indegree[course] == 0:
        queue.append(course)

# Count how many courses we are able to complete
completed_courses = 0

# Process courses in topological order
while queue:
    # Take a course with no remaining prerequisites
    current_course = queue.popleft()
    completed_courses += 1

    # Taking current_course may unlock other courses
    for next_course in graph[current_course]:
        # One prerequisite for next_course has now been completed
        indegree[next_course] -= 1

        # If next_course has no more prerequisites,
        # it can now be taken
        if indegree[next_course] == 0:
            queue.append(next_course)

# If we completed all courses, there was no cycle
# Otherwise, some courses were stuck in a cycle
return completed_courses == numCourses

```

4. Complexity Analysis

Let:

```

V = numCourses
E = len(prerequisites)

```

Time Complexity

```

O(V + E)

```

Explanation:

- We initialize arrays of size `V`.
- We build the graph using `E` prerequisite pairs.
- During BFS, each course is processed once.
- Each edge is processed once.

So the total time is:

```

O(V + E)

```

Space Complexity

```

O(V + E)

```

Explanation:

- The adjacency list stores `E` edges.
- The `indegree` array stores `V` values.
- The queue can store up to `V` courses.

So the total space is:

```

O(V + E)

```

5. Alternative Approaches

1. DFS Cycle Detection

We can also solve this using DFS with three states:

```
0 = unvisited
1 = visiting
2 = fully visited
```

During DFS:

- If we visit a node marked `1`, we found a cycle.
- If no cycle exists, all courses can be finished.

This is also:

```
Time:  O(V + E)
Space: O(V + E)
```

2. Topological Sort Using DFS

Instead of BFS/Kahn's Algorithm, we can perform DFS and build a topological ordering.

If a cycle is detected during DFS, return `False`.

Pattern Used

This is a classic:

```
Directed Graph Cycle Detection / Topological Sort
```

If a valid topological ordering exists, all courses can be finished. If not, the graph contains a cycle.